SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE:CSA1104

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 6 **Total Marks:60**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I __Dharshini/192521408__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Dharshini

Annexure A

ANALYTICAL PROBLEM SOLVING

1. Use Case Diagram Problem

A **Smart Banking System** allows customers to create accounts, transfer funds, pay bills, and view transaction history. Bank employees verify customer details, while administrators manage user accounts and banking services. The system requirements are only partially documented.

Tasks:

a) Identify all actors involved in the system.
b) List the primary use cases.
c) Define relationships (include, extend, generalization).
d) Construct a complete **Use Case Diagram** representing system functionality.

(10 Marks)

---

2. Class Diagram Problem

An **Online Vehicle Rental System** consists of entities such as Vehicle, Customer, Reservation, Payment, and RentalAgent. The relationships among these entities have not been clearly defined.

Tasks:

a) Identify classes with appropriate attributes and methods.
b) Define relationships (association, aggregation, inheritance, composition where applicable).
c) Specify multiplicity between classes.
d) Design a detailed **Class Diagram** for the system.

(10 Marks)

---

3. Sequence Diagram Problem

An **E-Commerce Shopping System** allows a customer to browse products, add items to the cart, place an order, make payment, and receive order confirmation. The sequence of interactions between system components is unclear.

Tasks:

a) Identify all objects participating in the interaction.
b) Define the message flow between objects.
c) Represent activation bars and lifelines.
d) Construct a **Sequence Diagram** showing the complete interaction flow.

(10 Marks)

---

4. Activity Diagram Problem

A **Job Recruitment Portal** enables applicants to register, upload resumes, apply for jobs, attend interviews, and receive offer letters. Recruiters verify applications before scheduling interviews. The business workflow needs optimization.

Tasks:

a) Identify all activities in the recruitment workflow.

b) Add decision nodes for selection and rejection.
c) Include parallel activities wherever applicable.
d) Design an optimized **Activity Diagram** representing the complete workflow.
(10 Marks)

---

5. State Diagram Problem

A **Smart Parking Management System** operates through states such as Parking Slot Available, Vehicle Entered, Parking Allocated, Payment Pending, Payment Completed, and Vehicle Exited. The system behavior during parking operations needs to be modeled.
Tasks:
a) Identify all possible states of the system.
b) Define events triggering state transitions.
c) Include initial and final states.
d) Construct a **State Transition Diagram** representing system behavior.
(10 Marks)

---

6. Deployment Diagram Problem

A **Smart Agriculture Monitoring System** uses IoT sensors to monitor soil moisture, temperature, and humidity. Sensor data is transmitted to a cloud platform where farmers monitor crop conditions through a mobile application. Automated irrigation is activated whenever soil moisture falls below a predefined threshold.
Tasks:
a) Identify all hardware nodes involved in the system.
b) Specify software components deployed on each node.
c) Define communication links between hardware nodes.
d) Construct a complete **Deployment Diagram** representing the system architecture.
(10 Marks)

Answer:

1. Use Case Diagram – Smart Banking System

(a) Identify the actors

- Customer: Creates accounts, transfers funds, pays bills, and views transaction history.
- Bank Employee: Verifies customer details.
- Administrator: Manages user accounts and banking services.

(b) Primary use cases

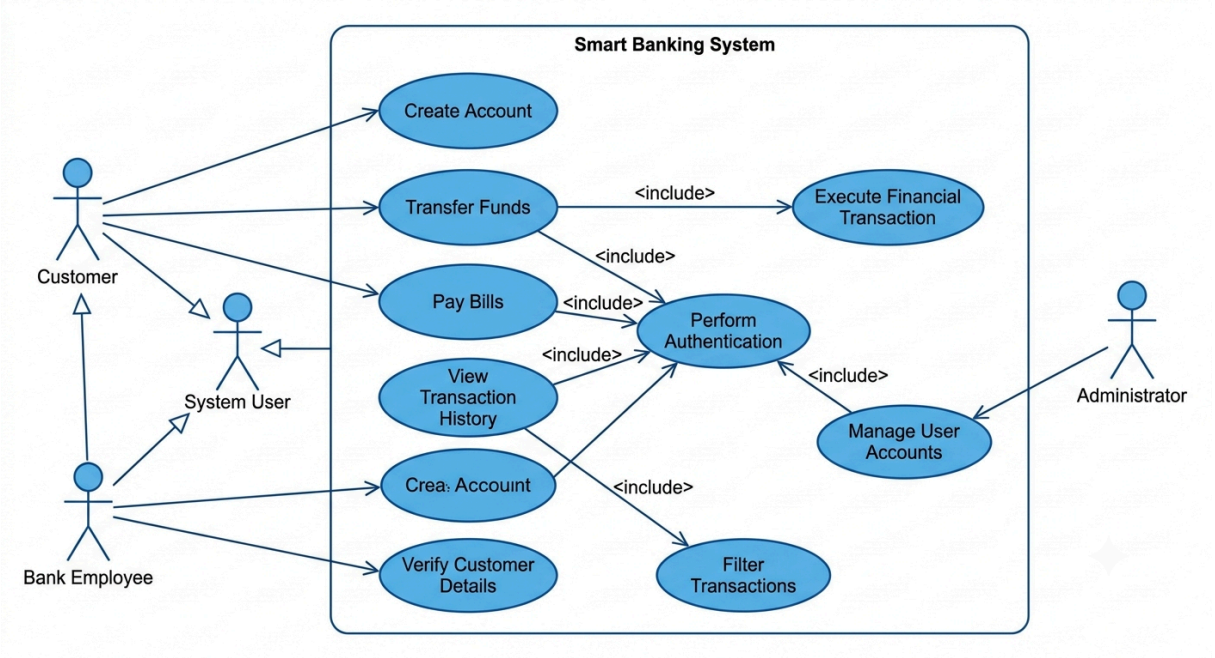
- Create Account
- Verify Customer Details
- Transfer Funds
- Pay Bills
- View Transaction History

- Manage User Accounts
- Manage Banking Services
- Authenticate User

(c) Relationships

- Include: `Transfer Funds` includes `Authenticate User` because authentication is required before transferring funds.
- Include: `Pay Bills` includes `Authenticate User`.
- Extend: `Verify Customer Details` can extend `Create Account` when verification is required during account creation.
- Generalization: The Customer and Bank Employee can be modeled as specialized types of a general `Bank User` actor if they share common authentication behavior.

(d) Use Case Diagram



The diagram shows how customers interact with banking services, employees verify customer information, and administrators manage the system. Authentication is included in fund transfers and bill payments to protect transactions.

2. Class Diagram – Online Vehicle Rental System

(a) Classes, attributes, and methods

Class	Attributes	Methods
Vehicle	vehicleId, model, registrationNo, availabilityStatus, rentalRate	checkAvailability(), updateStatus()
Customer	customerId, name, email, drivingLicense	searchVehicle(), makeReservation(), cancelReservation()
Reservation	reservationId, startDate, endDate, status	createReservation(), cancelReservation(), calculateCost()
Payment	paymentId, amount, paymentDate, paymentStatus	processPayment(), refundPayment()
RentalAgent	agentId, name, contactNo	verifyLicense(), approveReservation(), assignVehicle()

(b) Relationships

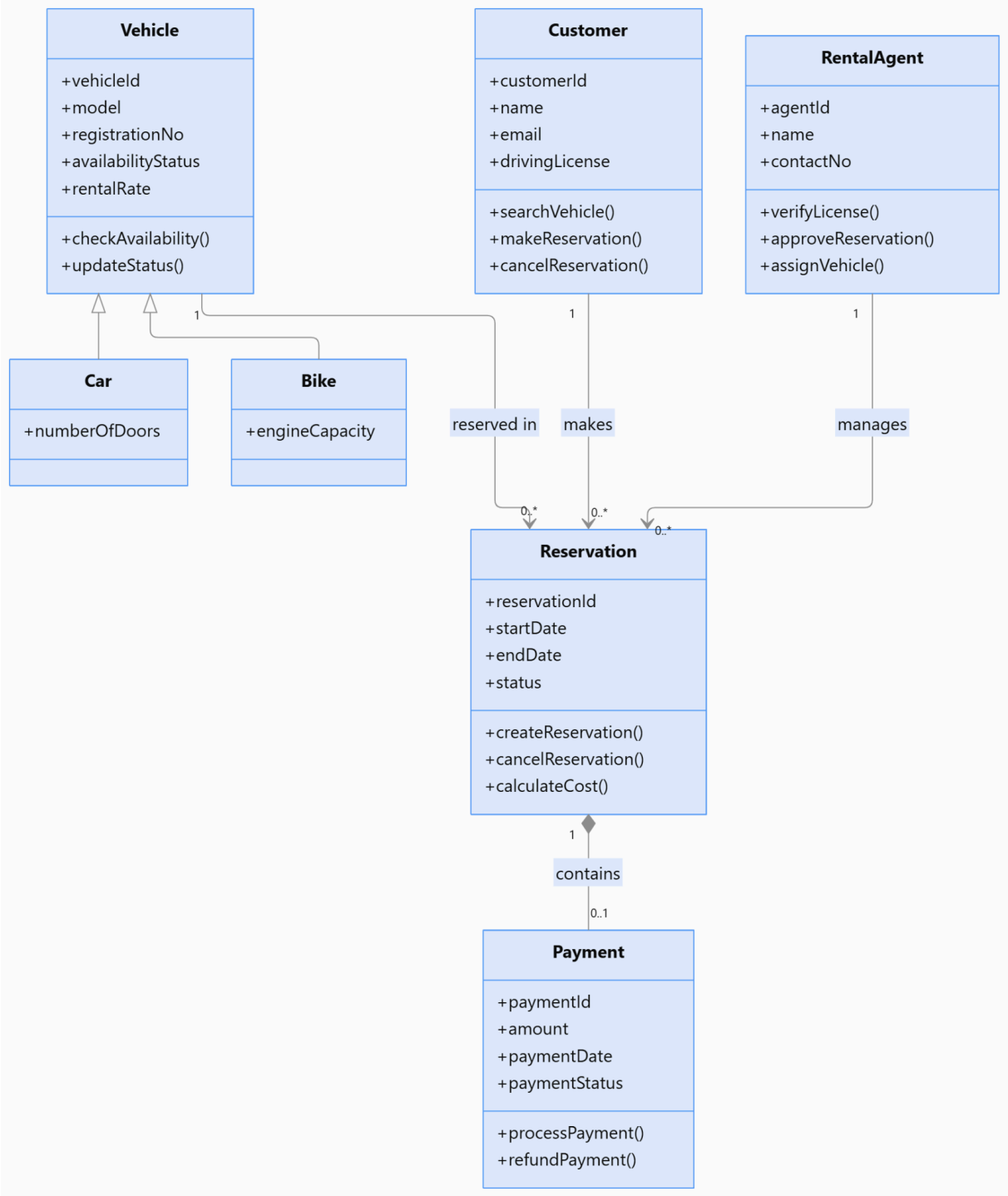
- Association: A Customer makes reservations, and a RentalAgent manages reservations.
- Aggregation: A RentalAgent can be associated with multiple Vehicles in a fleet-management arrangement; the vehicles can exist independently of the agent.
- Composition: A Reservation contains its associated Payment record in this simplified model; the payment record belongs to that reservation.
- Inheritance: `Car` and `Bike` can inherit common attributes and methods from `Vehicle`.

(c) Multiplicity

- One Customer can make zero or many Reservations: `Customer 1 – 0..* Reservation`.
- Each Reservation refers to one Vehicle, while a Vehicle can appear in many reservations over time: `Vehicle 1 – 0..* Reservation`.

- Each Reservation has zero or one Payment record: `Reservation 1 – 0..1 Payment`.
- One RentalAgent can manage many Reservations: `RentalAgent 1 – 0..* Reservation`.

(d) Class Diagram



The class diagram organizes the system into reusable classes. Inheritance allows Car and Bike to reuse Vehicle properties. Associations represent customer bookings, while composition links a reservation with its payment record.

3. Sequence Diagram – E-Commerce Shopping System

(a) Objects participating in the interaction

- Customer
- Shopping Interface
- Product Catalog
- Shopping Cart
- Order Service
- Payment Gateway
- Notification Service

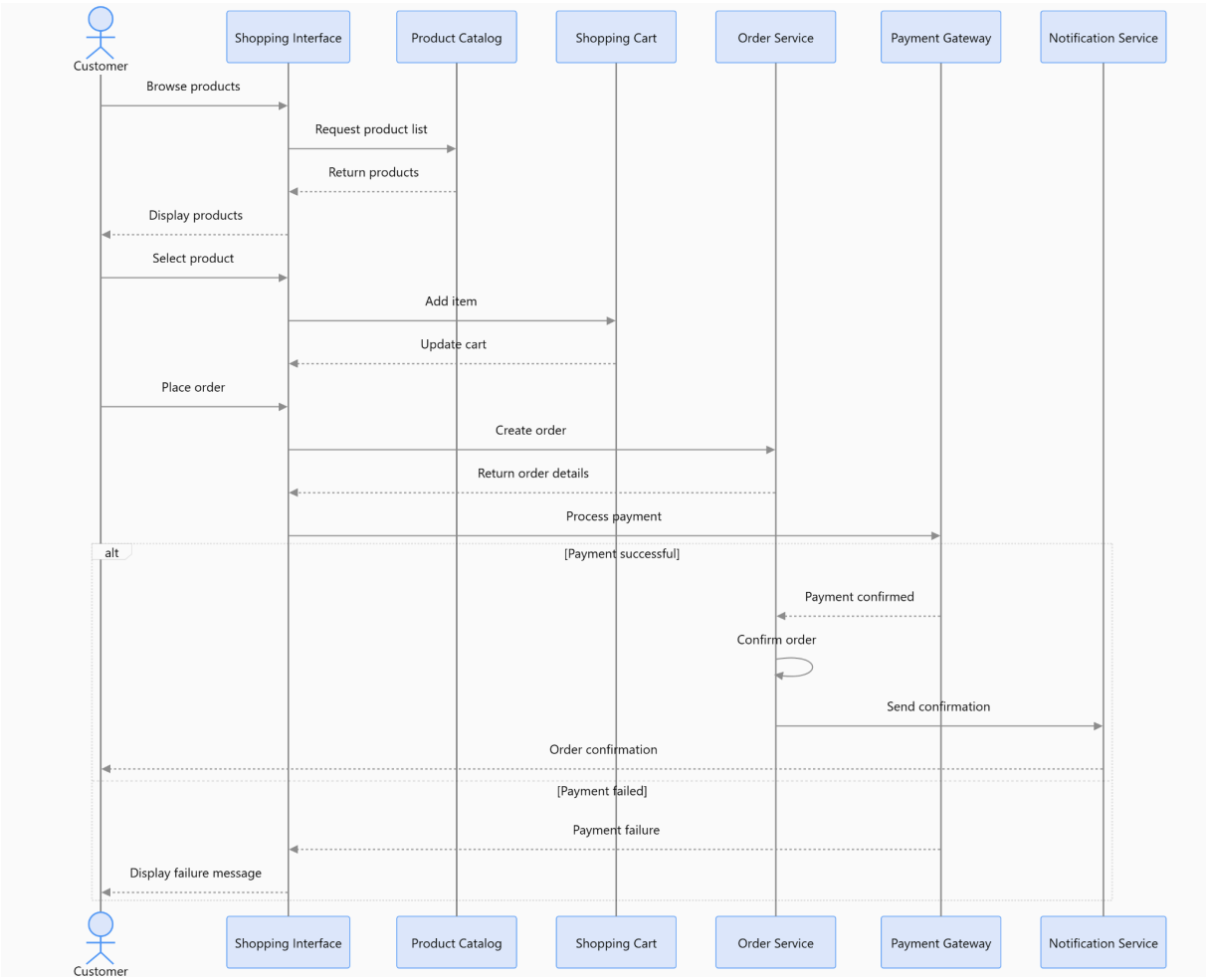
(b) Message flow

1. The customer opens the shopping interface.
2. The interface requests products from the Product Catalog.
3. The customer selects a product and adds it to the cart.
4. The customer proceeds to checkout.
5. The Order Service creates an order.
6. The Payment Gateway processes the payment.
7. If payment succeeds, the order is confirmed and a confirmation notification is sent.
8. If payment fails, the customer is informed and can retry payment.

(c) Activation bars and lifelines

- Lifelines represent the participating objects throughout the interaction.
- Activation bars indicate the period during which an object is executing an operation.
- Messages show requests and responses exchanged between objects.
- An alternative fragment (`alt`) represents the success and failure paths of payment.

(d) Sequence Diagram



The sequence diagram represents the time-ordered interaction between the customer and system components. It includes product browsing, cart updates, order creation, payment processing, and alternative outcomes for successful or failed payments.

4. Activity Diagram – Job Recruitment Portal

(a) Activities in the recruitment workflow

- Applicant registration
- Login and resume upload
- Job search and selection
- Job application submission
- Recruiter verification
- Application acceptance or rejection
- Interview scheduling
- Interview attendance
- Candidate evaluation
- Offer letter generation

- Final rejection notification

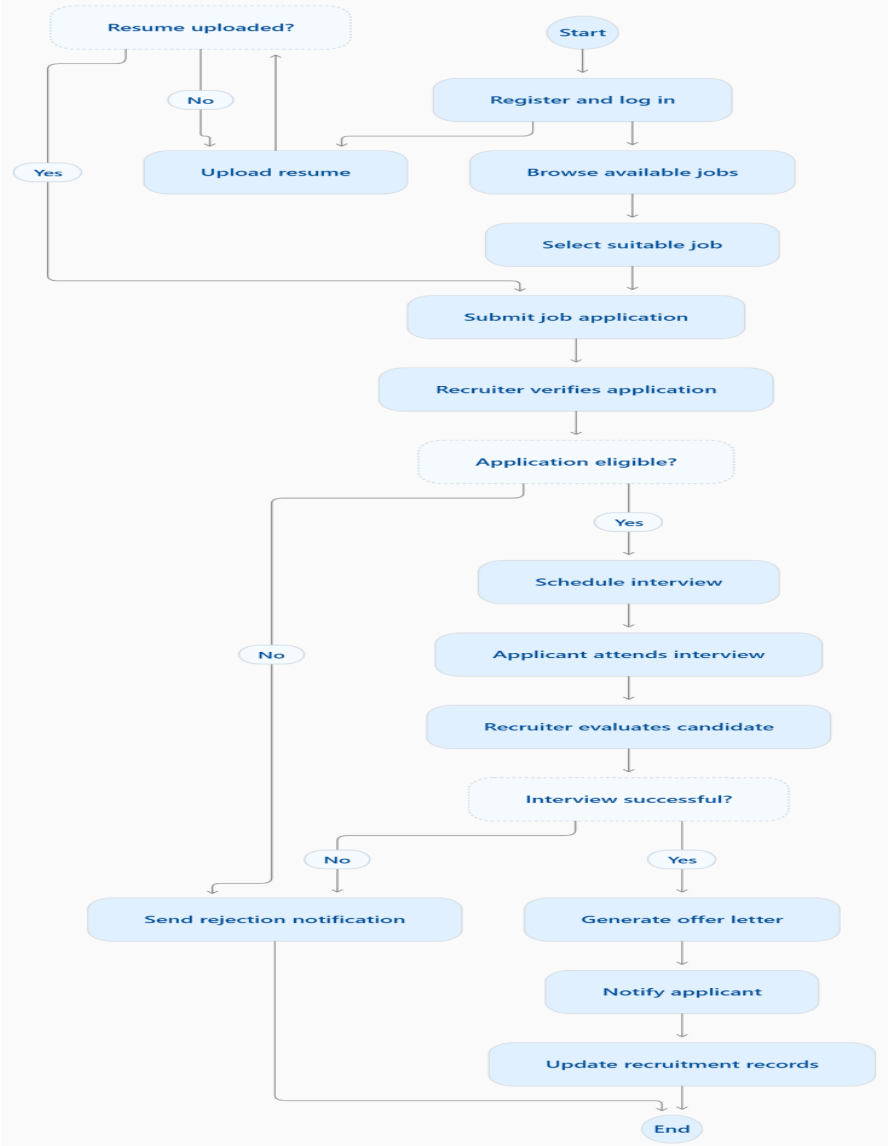
(b) Decision nodes

- Application eligible? — Determines whether the application proceeds to interview scheduling.
- Interview successful? — Determines whether the candidate receives an offer or a rejection notification.

(c) Parallel activities

After registration, the applicant can upload a resume and browse available jobs as independent activities. These activities can be performed in parallel before the applicant submits an application. After an offer is generated, the system can prepare the offer notification while updating the recruitment records.

(d) Activity Diagram



The activity diagram represents the recruitment process from registration to final selection. Decision nodes handle rejected applications and unsuccessful interviews. The process avoids scheduling interviews for ineligible candidates, reducing time and effort for recruiters.

5. State Diagram – Smart Parking Management System

(a) Possible states

1. Parking Slot Available: A parking space is free.
2. Vehicle Entered: A vehicle enters the parking area.
3. Parking Allocated: A suitable parking slot is assigned to the vehicle.
4. Payment Pending: The vehicle is parked, and payment is due.
5. Payment Completed: The parking fee has been paid successfully.
6. Vehicle Exited: The vehicle leaves the parking area and the slot becomes available again.

An additional state, Payment Failed, can be considered when a payment attempt is unsuccessful.

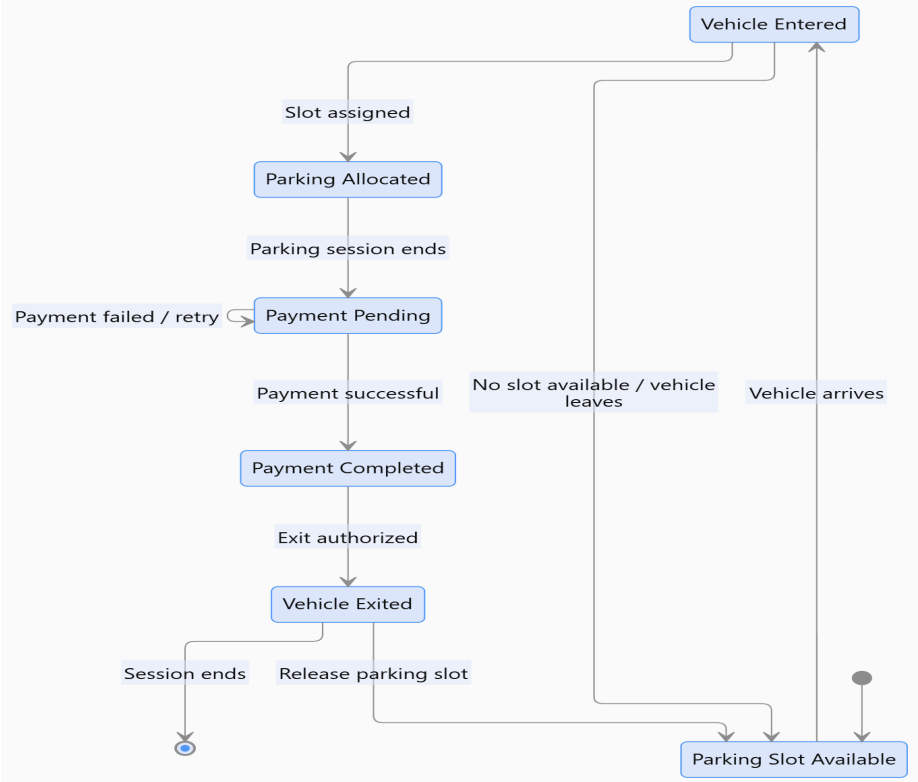
(b) Events triggering state transitions

- Vehicle arrival triggers the transition from Parking Slot Available to Vehicle Entered.
- Successful slot assignment triggers Parking Allocated.
- Parking completion or exit request triggers Payment Pending.
- Successful payment triggers Payment Completed.
- Exit authorization triggers Vehicle Exited.
- Slot release returns the system to Parking Slot Available.
- Failed payment keeps the vehicle in the payment process until the payment succeeds or an authorized alternative is used.

(c) Initial and final states

- Initial state: The system begins in Parking Slot Available.
- Final state: Vehicle Exited represents the end of one parking session. The parking management system itself continues operating.

(d) State Transition Diagram



The state diagram shows how the parking system changes states in response to vehicle movements and payment events. It ensures that a slot is allocated before parking, payment is completed before exit authorization, and the slot is released after the vehicle leaves.

6. Deployment Diagram – Smart Agriculture Monitoring System

(a) Hardware nodes

- IoT Sensor Nodes: Measure soil moisture, temperature, and humidity.
- Microcontroller/Gateway: Collects sensor readings and controls communication.
- Cloud Server: Receives, stores, and processes sensor data.
- Farmer's Mobile Device: Displays crop conditions through a mobile application.
- Irrigation Controller: Receives control commands to activate irrigation.
- Water Pump and Irrigation Equipment: Supply water to the crops.

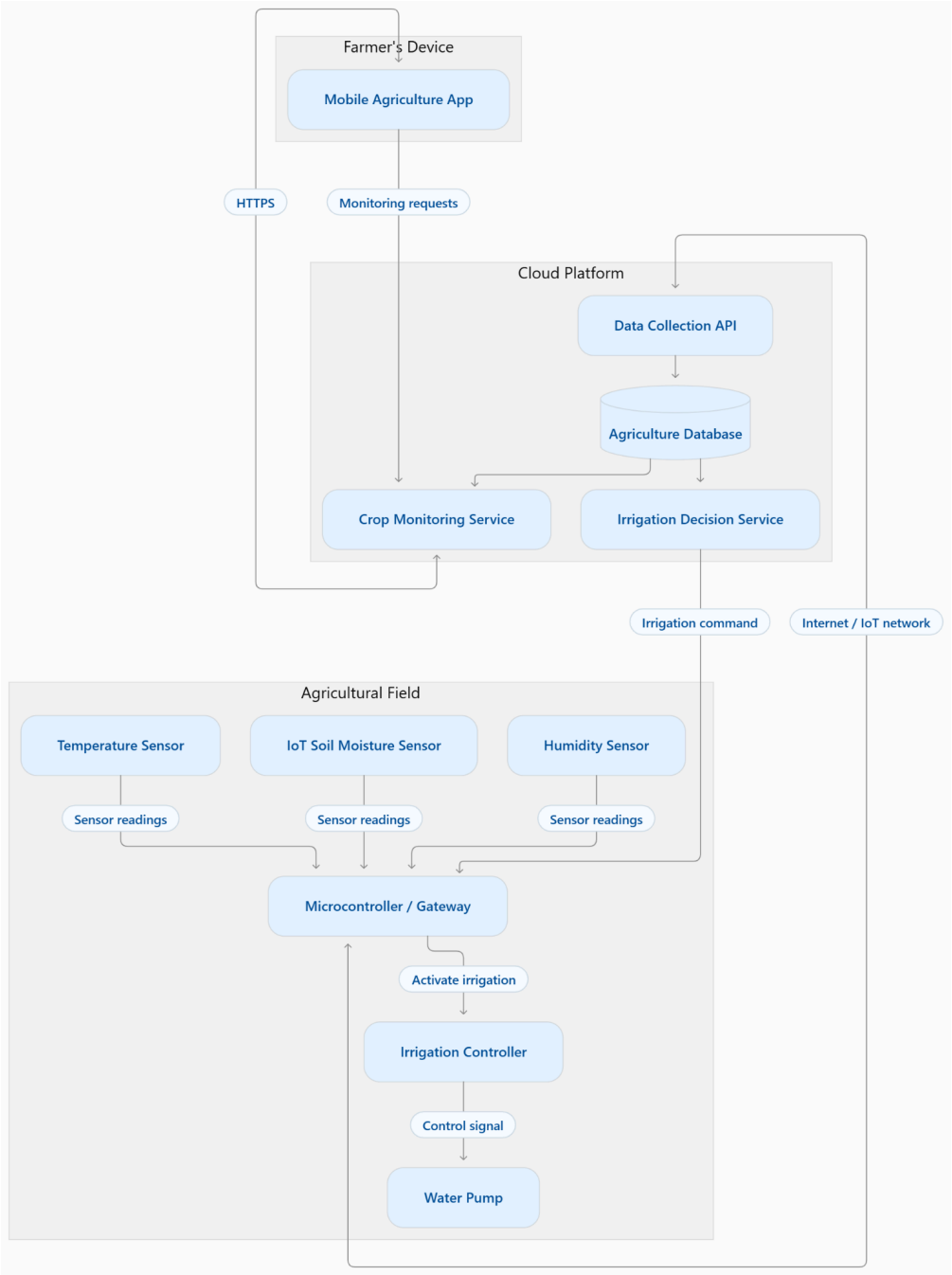
(b) Software components

Hardware Node	Software Components
IoT Sensor Nodes	Sensor drivers and data collection firmware
Microcontroller/Gateway	Data processing, communication, and threshold-checking firmware
Cloud Server	Data storage, monitoring service, and irrigation decision service
Farmer's Mobile Device	Agriculture monitoring application
Irrigation Controller	Pump-control firmware
Water Pump	Embedded control interface, where supported

(c) Communication links

- Sensors send readings to the gateway using a supported wireless protocol.
- The gateway transmits data to the cloud through Wi-Fi, cellular, or another available network.
- The mobile application communicates with the cloud through the internet.
- The irrigation decision service sends control commands to the irrigation controller.
- The controller activates the water pump when the soil moisture falls below the configured threshold.

(d) Deployment Diagram



The deployment diagram maps software components onto physical devices. Sensors collect environmental readings, the gateway forwards them to the cloud, and the farmer monitors crop conditions using a mobile application. When the soil moisture level falls below the predefined threshold, the irrigation decision service triggers the controller to activate the pump.

Conclusion: UML diagrams help represent the structure, interactions, workflows, state changes, and physical deployment of software systems. They make system requirements easier to understand and provide a clear blueprint for implementation.

RUBRICS FOR CALCULATION

ANALYTICAL PROBLEM SOLVING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation